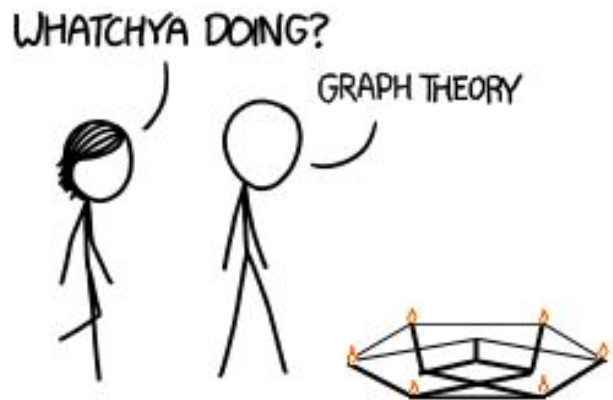


תכנות תחרותי

הרצאה 2



אז מה זה גרף בכלל??

גרף הוא מבנה מתמטי המשמש לייצוג קשרים.

לעיגולים קוראים צמתים (Vertices) ולקווים קשתות (Edges) ולפעמים יהיו על הקשתות גם משקלים.

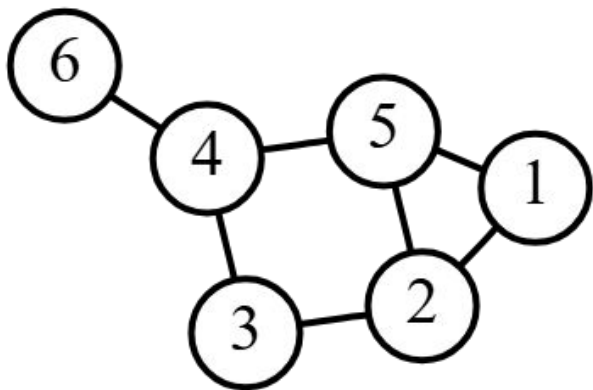
קיימים 2 סוגי גרפים עיקריים:

1. לא מכוונים - שבהם אם יש קשת בין צומת ראשונה לצומת שנייה, אז קיימת גם קשת בין השנייה לראשונה.

2. מכוונים - אם יש קשת בין צומת ראשונה לצומת שנייה, אז לא בטוח (אולי כן) קיימת קשת בין השנייה לראשונה.

סימון:

$$G(V,E) - V = \{\text{Vertices}\}, E = \{(u,v) \mid \text{Edges}\}$$

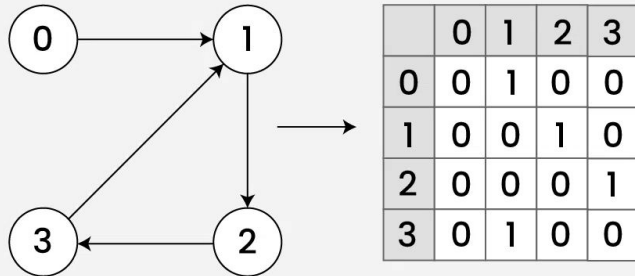


איך מייצגים גרפים?

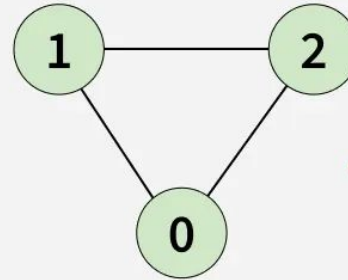
מטריצת שכנויות:

רשימת שכנויות:

**Adjacency Matrix for
Directed and Unweighted graph**



Adjacency Matrix A[]



Undirected Graph

Vertex	Neighbour	
0	→ 1	2
1	→ 0	2
2	→ 0	1

Adjacency List

רשימת שכנויות (מסמנים ב - $g[u]$)

```
int n,m;
cin>>n>>m; // n = number of vertices, m = number of edges
vector<vector<int>> adjacency_list(n);
for(int i=0;i<m;i++) { // input every edge
    int u,v;
    cin>>u>>v;
    u--, v--; // from vertex number to vertex index
    adjacency_list[v].push_back(u);
    // adjacency_list[u].push_back(v); - this is only for undirected graphs
}
int v; cin>>v;
vi neighbours = adjacency_list[v]; // all the neighbours of v
```

מטריצת שכנויות (לרוב לא משתמשים כי ריבועי)

```
int n;
cin>>n; // n = number of vertices
vector<vector<int>> adjacency_matrix(n, value:vector<int>(n));
for(int i=0;i<n;i++) {
    for(int j=0;j<n;j++) {
        cin>>adjacency_matrix[i][j]; // 1 or 0
    }
}
int v, u; cin>>v>>u;
adjacency_matrix[u][v]; // is there an edge (u,v)
```

סיורים בגרפים

סיורים בגרפים

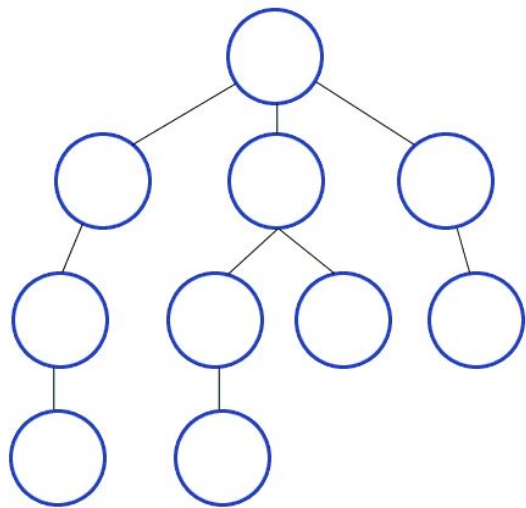
בדומה למערכים, לפעמים נרצה לעבור על גרפים ולבצע עליהן פעולות, אבל בשונה ממערכים לגרפים אין סדר מוגדר שאפשר לעבור עליהם.

לכן המציאו 2 שיטות סיור עיקריות בגרפים:

1. DFS - Depth-first search
2. BFS - Breadth-first search

DFS

הסיור יתחיל מצומת כלשהי s , ונסמן כל צומת שביקרנו בה כך שלא נבקר בצומת פעמיים.
כאשר נגיע לצומת u - נסמן אותה ולאחר מכן נבקר רקורסיבית בכל v ב- $g[u]$ שהם השכנים של u .



סיבוכיות - $O(V+E)$

מקום - $O(V+E)$

תרגיל: תיקחו 5 דקות ותכתבו קוד ל DFS - ב - cpp


```
vi vis(n, value:0); // n - number of vertices
void dfs(vector<vector<int>> &g, int u) {
    // we get a pointer to g because otherwise "vector<vector<int>> g" will copy g every dfs call - bad runtime.
    vis[u] = 1;
    // do action
    for(int v : g[u]) {
        if(vis[v])
            continue;
        dfs(&g, v);
    }
}
```

BFS

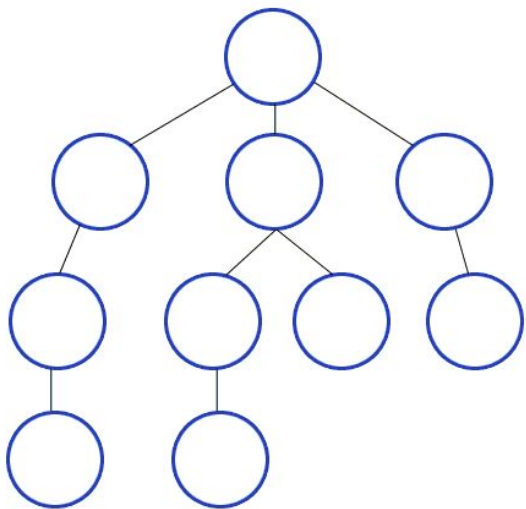
הסיור יתחיל מצומת כלשהי s , ונתפשט ממנו בסדר ממזין לפי מרחק (בשכבות).

נתחזק תור שיתחיל עם s , וכל פעם נוציא את הצומת שבראש התור כל עוד התור לא ריק נכניס את השכנים שלו לסוף התור (אם לא בוקרו) ונסמן שביקרנו בהם.

סיבוכיות - $O(V+E)$

מקום - $O(V+E)$

תרגיל: הפעם לכתוב קוד ל - BFS



```
void bfs(vector<vector<int>> &g, int u) {  
    // again we get a pointer to g  
    vis[u] = 1;  
    queue<int> q;  
    q.push(u);  
    while(!q.empty()) {  
        int u = q.front();  
        q.pop();  
        // do action  
        for(int v : g[u]) {  
            if(vis[v])  
                continue;  
            q.push(v);  
            vis[v] = 1;  
        }  
    }  
}
```

BFS vs DFS

יש המון יתרונות לשימוש ב-2 האלגוריתמים ← אבל! לרוב בבעיות של גרפים לא ממושקלים אנחנו נשתמש בעיקר ב-BFS, אך קיימים מקרים כמו שלא משנה באיזה סיור נשתמש - אז נעשה DFS כי הוא יותר קל למימוש, או שימושים אחרים כמו הסתכלות על גרף כמו עץ, ועוד.

ותמיד אפשר להוסיף עוד פרמטרים ופעולות לפונקציות בהתאם לדרישת השאלה - לדוגמא חישוב מרחק, עומק, כמות ילדים וכו'.

בעיית המסלול הקצר ביותר

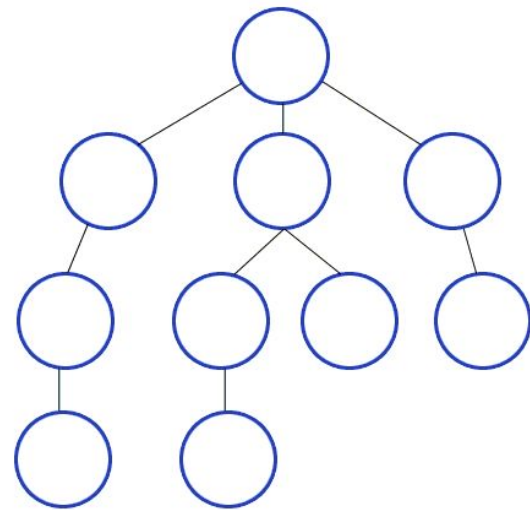
בעיית המסלול הקצר ביותר - גרף חסר משקלים

בעיה: נרצה לחשב את המרחק הקטן ביותר מצומת נתונה v לכל צומת אחרת בגרף
בזמן $O(V+E)$!

תחשבו רגע איך פותרים?

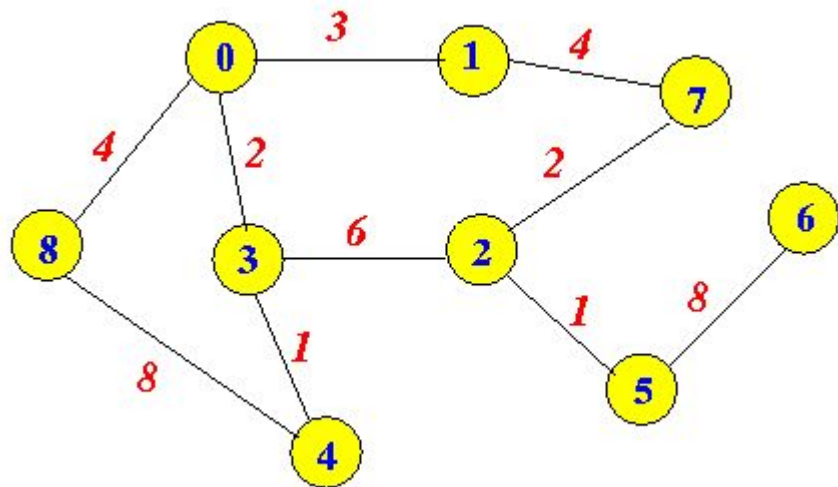
פתרון: פשוט נריץ את BFS!

```
void bfs(vector<vector<int>> &g, int u, vector<int> &dist) {  
    dist[u] = 0;  
    vis[u] = 1;  
    queue<int> q;  
    q.push(u);  
    while(!q.empty()) {  
        int u = q.front();  
        q.pop();  
        for(int v : g[u]) {  
            if(vis[v])  
                continue;  
            q.push(v);  
            dist[v] = dist[u] + 1;  
            vis[v] = 1;  
        }  
    }  
}
```



בעיית המסלול הקצר ביותר - גרף ממושקל

בעיה: נרצה לחשב את המרחק הקטן ביותר מצומת נתונה v לכל צומת אחרת בגרף
מה ההבדל? הפעם לכל קשת יש משקל!



בדומה לגרף רגיל, גרף ממושקל מיוצג על ידי רשימת שכנויות $g[u]$ של זוגות $\{v, w\}$

איך מוגדר המרחק הקטן ביותר?

Bae: Come over

Dijkstra: But there are so many routes to take and I don't know which one's the fastest

Bae: My parents aren't home

Dijkstra:

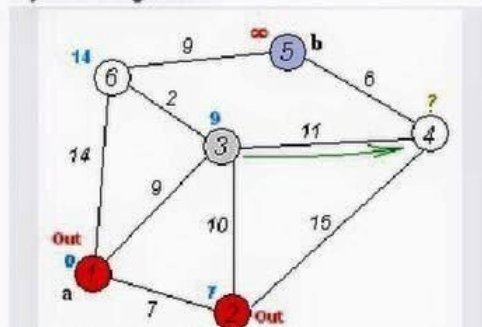
Dijkstra's algorithm

Graph search algorithm

Not to be confused with Dykstra's projection algorithm.

Dijkstra's algorithm is an **algorithm** for finding the **shortest paths** between **nodes** in a **graph**, which may represent, for example, road networks. It was conceived by computer scientist **Edsger W. Dijkstra** in 1956 and published three years later.^{[1][2]}

The algorithm exists in many variants, Dijkstra's original variant found the shortest path between two nodes,^[2] but a more common variant fixes a single node as the "source" node and finds shortest paths from the source to all other nodes in the graph, producing a **shortest-path tree**.



אלגוריתם דייקסטרה `dijkstra`

הרעיון הוא להבין שאם נסתכל על כל השכנים של צומת ההתחלה s , זה בעל משקל קשת מינימלי הוא בהכרח הדרך הכי יעילה להגיע מ- s אליו.

למה: אם נסתכל על קבוצת k הצמתים הכי קרובים לשורש s , הדרך הכי קצרה מ- s לצומת שאיננו בקבוצה זו, נסמן אותו v , היא בהכרח הדרך הכי קצרה מהשורש s אל v , בנוסף v הוא הצומת ה- $k+1$ הכי קרוב ל- s .

אלגוריתם דייקסטרה *dijkstra*

אלגוריתם:

נתחיל מלהגדיר את המרחק של s ל- 0 כי כמובן שהמרחק של s מעצמו הוא 0 . אנחנו יודעים כי $A = \{s\}$ הוא הקבוצה בגודל 1 של צמתים הכי קרובים ל- s וכל המסלולים החוצה מהקבוצה הם הקשתות של s .

נתחזק priority-queue של משקלי כל המסלולים החוצה מ- A (בהתחלה השכנים של s). כל פעם ניקח את המסלול בעל המשקל הקטן ביותר, נגדיר לצומת v אליו מוביל המסלול את המשקל שלה בתור משקל המסלול ונוסיף את כל הקשתות החוצה מ- v בתור מסלול בעל המשקל של v ועוד משקל הקשת.

אלגוריתם דייקסטרה dijkstra

```
void dijkstra(int n, vector<vector<pii>> &g, int s, vector<int> &dist) {
    rep(i,0,n) dist[i] = -1; //start by setting the distances of every vertex to -1

    //create a priority queue and start by inserting a route leading to s with a cost of 0
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, s});

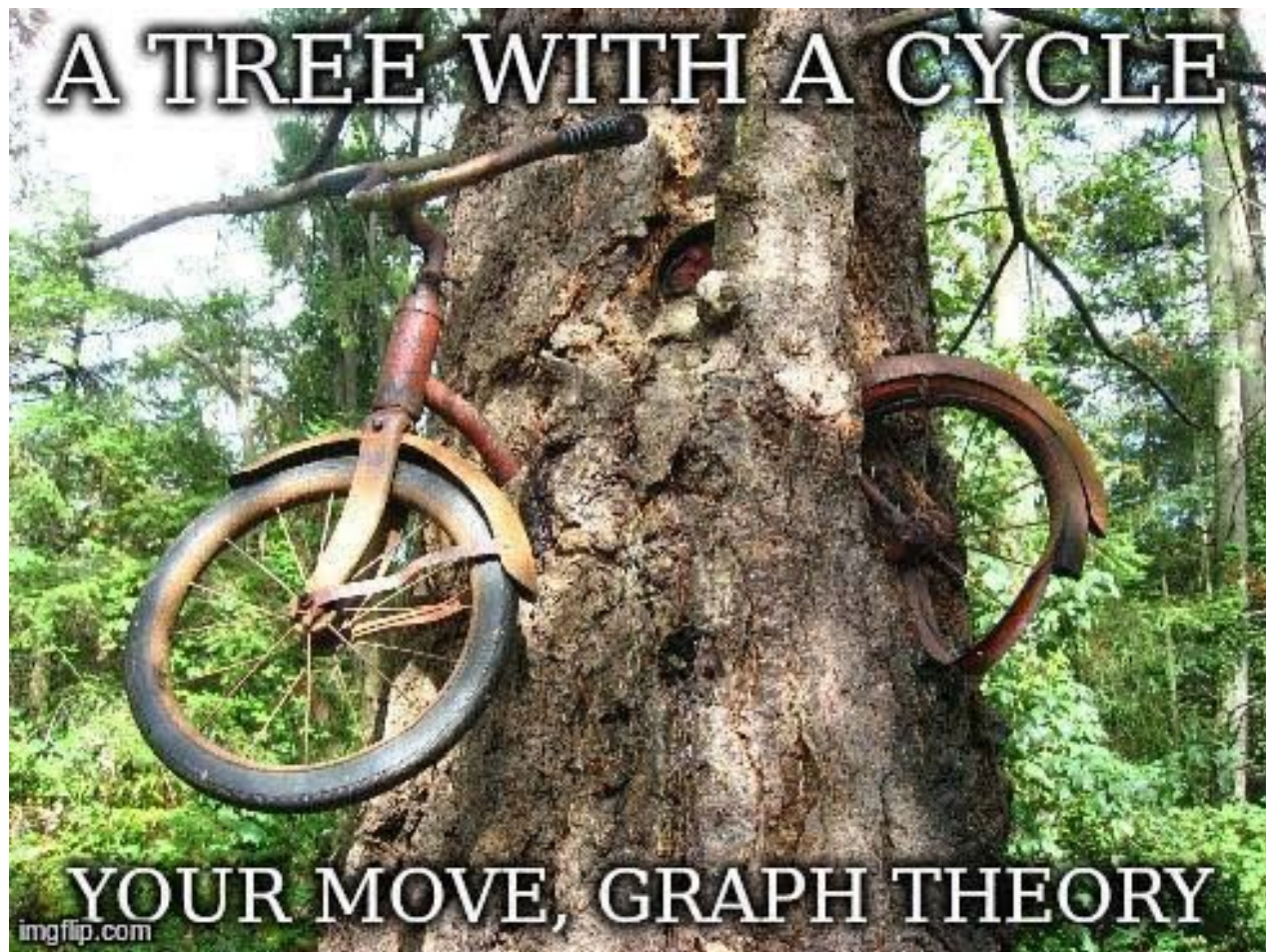
    //while the queue isn't empty look at the shortest route in the queue
    while(!pq.empty()) {
        pii p = pq.top(); pq.pop();

        //check if the route is to a vertex we have not seen yet, AKA outside of A
        if (dist[p.second] != -1) continue;

        //set the distance of the new discovered vertex to the weight of the path
        dist[p.second] = p.first;

        //add the new paths from the new vertex to the queue
        for(auto v : g[p.second]) {
            pq.push({dist[p.second] + v.first, v.second});
        }
    }
}
```

A TREE WITH A CYCLE

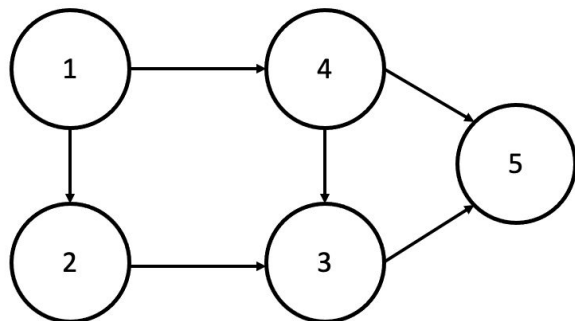


YOUR MOVE, GRAPH THEORY

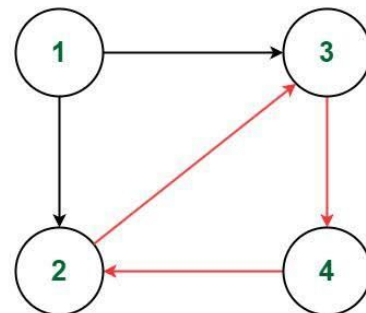
DAG - Directed Acyclic Graph

DAG או בעברית גמל - גרף מכוון חסר מעגלים כשמו כן הוא, גרף שבו הקשתות מכוונות (קשת בין a ל-b לא אומרת שיש קשת בין b ל-a) שבו אין מעגלים **מכוונים**

מעגל מכוון הוא מסלול בין הקשתות u אל הקשתות u בעל לפחות קשת אחת



גרף בלי מעגל מכוון

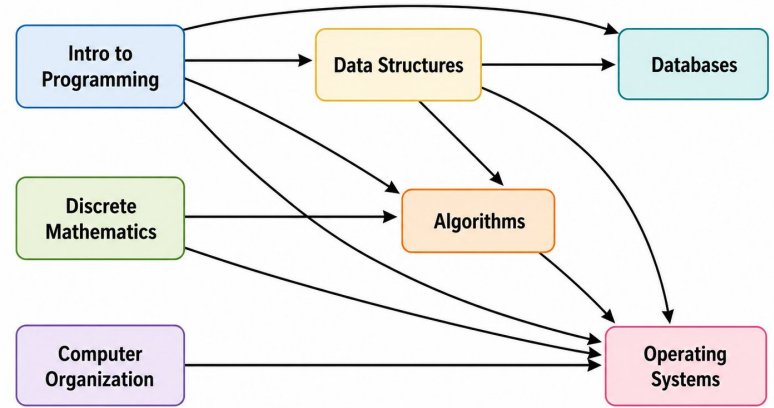


גרף עם מעגל מכוון

IMCOM Command Structure

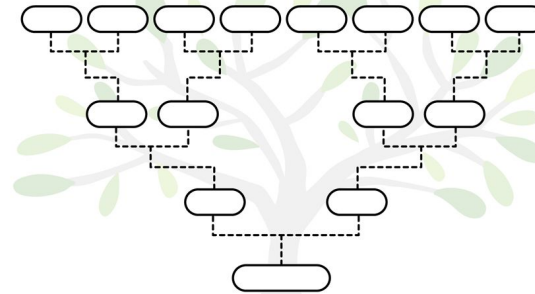


Example DAG: Course Prerequisites



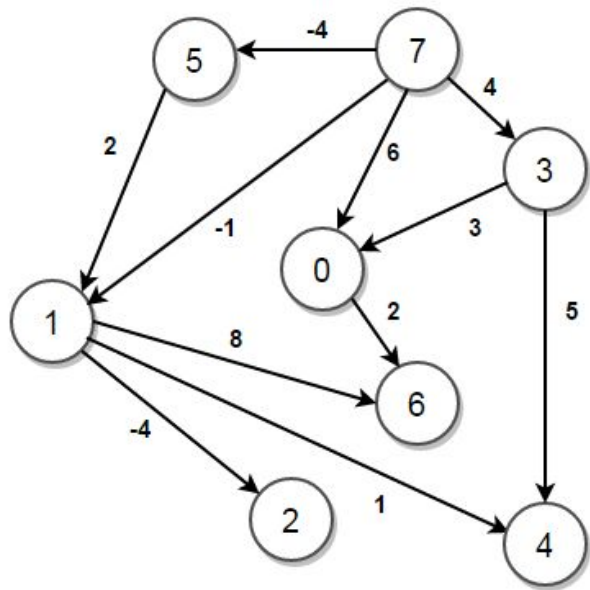
→ A → B means "A is a prerequisite for B"

Family Tree

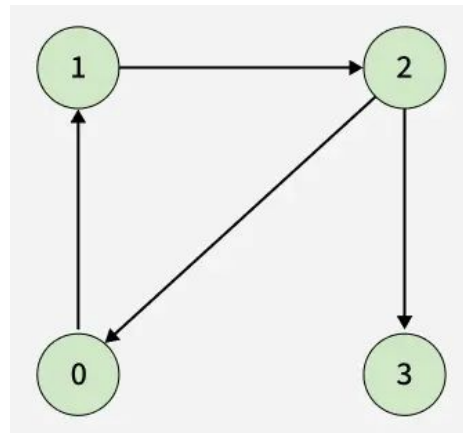


מה משותף לכל הגרפים האלה?

נשים לב שבכל הגרפים האלה יש סוג של היררכיה, קל לראות איזה צומת הגיע לפני איזה צומת. התכונה הזו תמיד קיימת ב-DAG גם אם לפעמים קשה יותר לראות אותה בעין.



חשוב לציין שהתכונה הזו ייחודית לגרפים חסרי מעגלים, אם יש מעגל לא נוכל להגיד מי הגיע קודם.



מיון טופולוגי

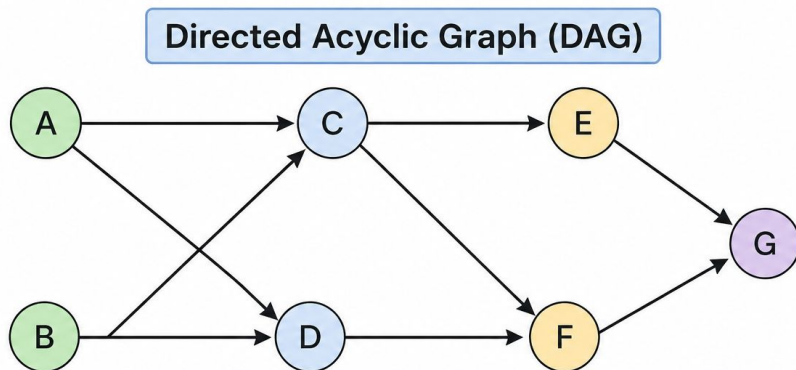
מיון טופולוגי הוא חלוקה של הגרף לשכבות. במילים אחרות, מספור של הצמתים בגרף כך שלכל שני צמתים u, v שממוספרים אם יש קשת מ- u ל- v אז $u > v$

מיון טופולוגי

מי הצומת הראשון שלנו בעצם?

בעץ יש לנו שורש, אבל לא כל גרף חסר מעגלים הוא עץ

הצמתים הראשונים הם הצמתים שאין להם הורה!



מיון טופולוגי

איך מוצאים מיון טופולוגי לגרף?

קחו כמה דקות לחשוב על אלגוריתם

מיון טופולוגי

איך מוצאים מיון טופולוגי לגרף?

פתרון:

כל הצמתים ללא הורה כלומר בעלי דרגת כניסה 0 נמצאים בשכבה הראשונה. נוריד אותם מהגרף ונסתכל שוב פעם על הצמתים שעכשיו אין להם הורה, הם בשכבה השניה, וכך הלאה.

נשים לב שבגרף מכוון חסר מעגלים תמיד יש צומת בלי הורה. בנוסף אם הורדתי צומת, הצמתים היחידים שיכול להיות שפתאום אין להם יותר הורה הם הבנים של אותה צומת.

מיון טופולוגי

```
void solve(){
    int n, m, a, b;
    cin >> n >> m;
    queue<int> q;
    vector<int> in(n, value: 0);
    vector<vector<int>> g(n);
    for(int i = 0; i < m; i++){
        cin >> a >> b; a--; b--;
        g[a].push_back(b);
        in[b]++;
    }
    for(int i = 0; i < n; i++) if(in[i] == 0){
        q.push(x: i);
    }
    while(!q.empty()){
        int v = q.front();
        q.pop();
        for(auto u : long long : g[v]){
            //do something
            in[u]--;
            if(in[u] == 0) q.push(x: u);
        }
    }
}
```

שאלה לדוגמא

- A game has n levels, connected by m teleporters, and your task is to get from level 1 to level n . The game has been designed so that there are no directed cycles in the underlying graph. In how many ways can you complete the game?

פתרון

נשים לב שכמות הדרכים להגיע לצומת מסויים הוא סכום כמות הדרכים להגיע לכל ההורים הישירים שלו (צמתים שיש מהם קשת אליו)

נאתחל הצומת הראשון את כמות הדרכים להגיע אליו ל-1 ונעבור בסדר של המיון הטופולוגי על הצמתים כדי לבסוף לגלות את כמות הדרכים להגיע ל-n

פתרון

```
const int md = 1e9+7;

void solve(){
    int n, m, a, b;
    cin >> n >> m;
    queue<int> q;
    vector<int> in(n, value: 0), ways(n, value: 0);
    vector<vector<int>> g(n);
    for(int i = 0; i < m; i++){...} //input
    for(int i = 0; i < n; i++) if(in[i] == 0){
        q.push(x: i);
    }
    ways[0] = 1;
    while(!q.empty()){
        int v = q.front();
        q.pop();
        for(auto u : long long : g[v]){
            ways[u] += ways[v];
            ways[u] %= md;
            in[u]--;
            if(in[u] == 0) q.push(x: u);
        }
    }
    cout << ways[n-1];
}
```



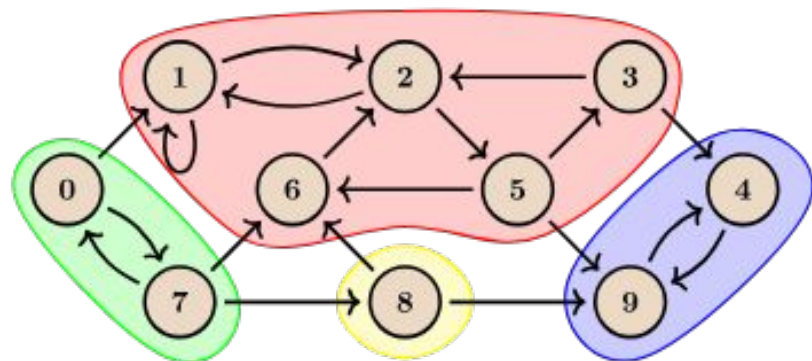

SCC - Strongly Connected Component



מה זה רכיב קשירות חזק?

קשירות חזקה: בין 2 צמתים u, v יש קשירות חזקה $\leftarrow$ קיים מסלול היוצא מ- u ומגיע ל- v , וגם קיים מסלול היוצא מ- v ומגיע ל- u .

רכיב קשירות חזק: בגרף מכוון אוסף של קודקודים יקרא רכיב קשירות חזק $\leftarrow$ בין כל זוג קודקודים u, v השייכים לרכיב קיימת קשירות חזקה.



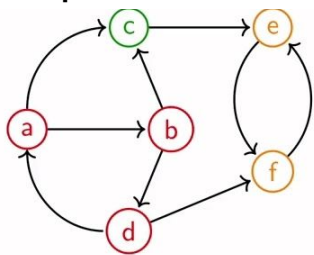
איך מוצאים דבר שכזה?

אבחנה:

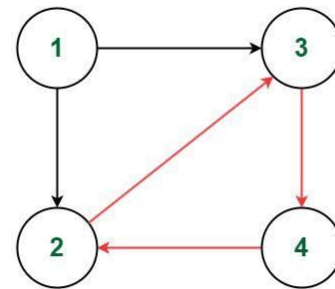
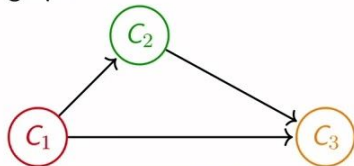
אבחנה: נשים לב כי אם נדחוס כל רכיב קשירות חזק להיות קודקוד אחד נקבל DAG.

אינטואיציה להוכחה: מניחים בשלילה כי קיים מעגל בגרף הדחוס - כלומר הגרף אינו DAG, נגיד המעגל מגודל 3 כמו בדוגמא: אז כל צומת ב-4 יכול להגיע לכל צומת ב-2, כל צומת ב-2 יכול להגיע לכל צומת ב-3 וכל צומת ב-3 יכול להגיע לכל צומת ב-4. לכן נוכל ליצור מסלול מ-4 שעובר דרך קודקודים ב-2 ומגיע ל-3 ולכן כל קודקוד מ-4 יכול להגיע לכל קודקוד ב-3. וגם כל קודקוד ב-3 יכול להגיע לכל קודקוד ב-4. סתירה לכך שהם רכיבי קשירות חזקים שונים.

דחיסת הגרף:



$SCC(G)$ of the above directed graph G is



דוגמא
להוכחה:

אלגוריתם למציאת SCC

1. Run dfs from some vertex s , mark s and every other vertex you see, and when some vertex v finish running - push v in a stack.

(if the whole graph is not visited we run step 1 again from an unvisited vertex)

2. Reverse the graph // every edge (u,v) is now (v,u)
3. Create vector to remember the SCC index, and a current $SCC = 0$;
4. Run dfs by order from the stack only if stack top isn't indexed yet.
every time we run a new dfs we will start a new SCC ($current++$) and we will save the new index for every vertex we reach with the current dfs

אינטואיציה לאלגוריתם

לרגע נתעלם מהצעדים הראשונים. מהצעד האחרון אנחנו בודקים עבור קודקוד v אילו קודקודים בגרף המקורי היה מסלול מהם אל v .

אנו נרצה להריץ את ה-dfs השני כך שהוא לא "יברח" לרכיב קשירות אחר. מאחר והגרף הדחוס הוא DAG - רכיב הקשירות של קודקוד s הוא עם דרגת כניסה 0, ולכן יהיה עם דרגת יציאה 0 בגרף ההפוך. אז שנריץ dfs על s נקבל רכיב קשירות חזק.

ואז אם נוריד את רכיב הקשירות של s , יהיה לנו רכיב קשירות חדש עם דרגת כניסה 0 ש- s הצביע אליו.

Graph of SCCs



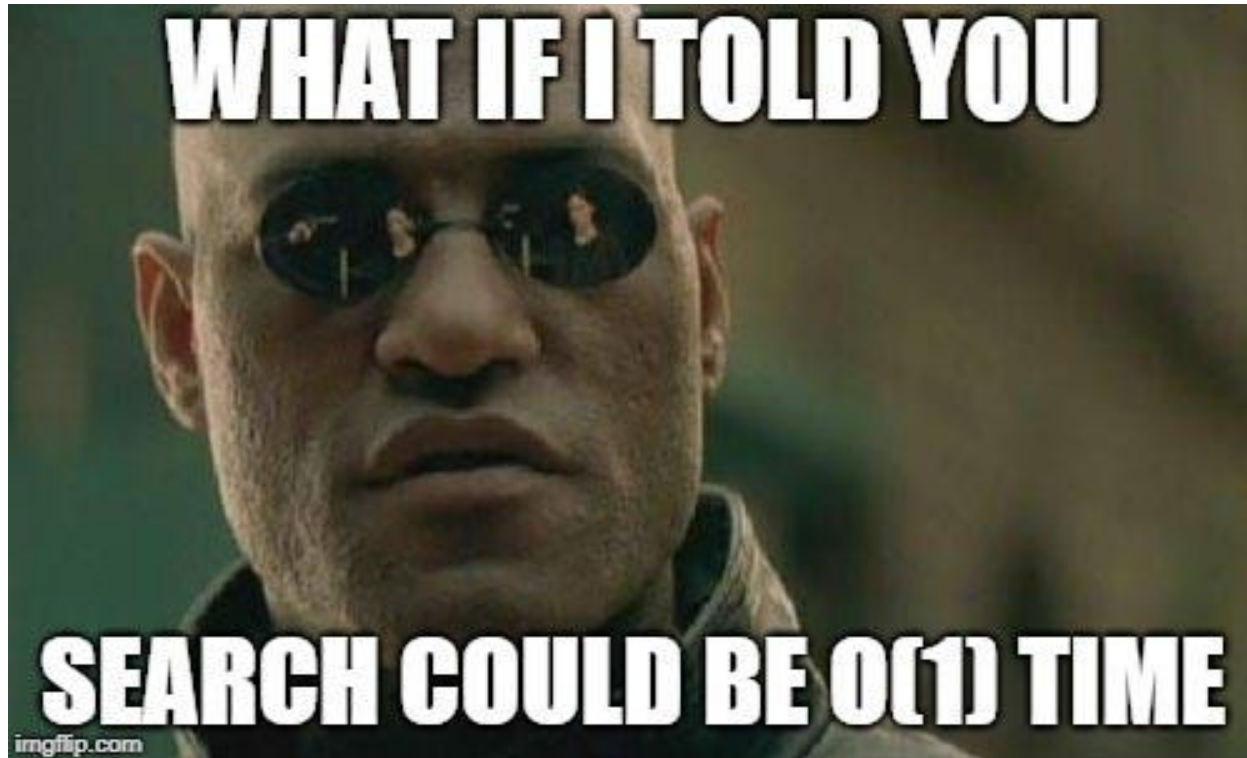
SCCs in reverse graph



קוד (יהיה לכם בתחרות)

```
namespace SCCKosaraju {
    vector<vector<int>> adj, radj;
    vector<int> todo, comp;
    vector<bool> vis;
    void dfs1(int x) {
        vis[x] = 1;
        for (int y : adj[x])
            if (!vis[y]) dfs1(y);
        todo.push_back(x);
    }
    void dfs2(int x, int i) {
        comp[x] = i;
        for (int y : radj[x])
            if (comp[y] == -1) dfs2(y, i);
    }
    vector<int> scc(vector<vector<int>>& _adj) {
        adj = _adj;
        int time = 0, n = adj.size();
        comp.resize(n, -1), radj.resize(n), vis.resize(n);
        for (int x = 0; x < n; x++)
            for (int y : adj[x]) radj[y].push_back(x);
        for (int x = 0; x < n; x++)
            if (!vis[x]) dfs1(x);
        reverse(todo.begin(), todo.end());
        for (int x : todo)
            if (comp[x] == -1) dfs2(x, time++);
        return comp;
    }
} namespace SCCKosaraju
```

UNION-FIND



כמו שכולנו כבר מכירים

יש לנו קבוצות של n איברים ונוכל לבצע 2 פעולות:

1. איחוד 2 קבוצות ב $O(\alpha)$
2. בהינתן איבר - באיזו קבוצה הוא נמצא ב $O(\alpha)$

בגלל שאנחנו לא אנשי תיאוריה אז:

$$O(\alpha) = O(1)$$

קוד קל ופשוט

```
const int maxn = 2e5+5; //change it to be the size of the group
int p[maxn], s[maxn];
void pre(){
    rep(i, 0, maxn) p[i] = i, s[i] = 1;
}
int find(int a) { return a == p[a] ? a : p[a] = find(p[a]); }
int union(int a, int b) {
    if((a = find(a)) == (b = find(b))) return 0;
    if(s[a] < s[b]) swap(&a, &b);
    return s[a] += s[b], p[b] = a, 1;
}
```

כמה שאלות לסיכום

Time limit: 1.00 s **Memory limit:** 512 MB

Your task is to find a minimum-price flight route from Syrjälä to Metsälä. You have one discount coupon, using which you can halve the price of any single flight during the route. However, you can only use the coupon once.

When you use the discount coupon for a flight whose price is x , its price becomes $\lfloor x/2 \rfloor$ (it is rounded down to an integer).

Input

The first input line has two integers n and m : the number of cities and flight connections. The cities are numbered $1, 2, \dots, n$. City 1 is Syrjälä, and city n is Metsälä.

After this there are m lines describing the flights. Each line has three integers a , b , and c : a flight begins at city a , ends at city b , and its price is c . Each flight is unidirectional.

You can assume that it is always possible to get from Syrjälä to Metsälä.

Output

Print one integer: the price of the cheapest route from Syrjälä to Metsälä.

Constraints

- $2 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$
- $1 \leq c \leq 10^9$

Example

Input:

```
3 4
1 2 3
2 3 1
1 3 7
2 1 5
```

Output:

```
2
```

זהו להיום!